# Public form spam protections & improvements

Repository assessment | 7 October 2026 | Commit 876070a

Existing controls stop a small burst from the same rate-limit key. They do not reliably stop repeated submissions across changed identities, multiple application processes, or direct API clients. Keep forms available without customer login and add enforcement at the API and edge.

## Highest-priority findings

| Priority | Finding and consequence |
|---|---|
| High | Career submission limits include the supplied email. Changing email creates a fresh quota for the same IP. Confirmed with a local helper-level reproduction. |
| High | Ordinary submission counters live in process memory. Every worker has its own quota; restarts clear it. Distributed traffic can exceed the intended allowance. |
| Medium | No CAPTCHA verification, honeypot, recipient email quota, or general submission idempotency was found in the reviewed form paths. Valid-looking automated requests can create records and send acknowledgements. |
| Conditional high | Generic and bespoke core create routes remain generated. If create permission is granted to Public or a reachable service token, those routes can bypass custom submit checks. Runtime permissions were not inspected. |

## Assessment boundary

Inspected CMS routes, submission controllers, rate-limit helpers, middleware ordering, trusted-customer policy, email senders and targeted tests. Frontend handlers, deployed nginx/CDN rules, firewall state, live environment values and Strapi role/API-token permissions were unavailable. The numbers below are source defaults, not verified production settings.

No application code was changed. No production submissions, email sends, dependency audit or load tests were performed. This report distinguishes confirmed code behavior from deployment-dependent exposure.

# 01 / Current guardrails

All routes below are POST requests under the default /api prefix. Ordinary submission limits default to 5 attempts per 60 minutes, with HTTP 429 and Retry-After when exhausted. Validation failures reaching the limiter also consume quota.

| Route / shopper access | Current enforcement |
| --- | --- |
| /generic-submissions/submit Public: auth false | Memory counter: generic + IP + formTag. Published form lookup, configured required fields and consent, basic contact/date validation, uploaded files rejected. |
| /bespoke-submissions/submit Public: auth false | Memory counter: bespoke + IP. Published form must be enabled; required contact/design fields; optional image MIME check and 5 MiB file check. |
| /submissions-job-openings/submit Public: auth false | Memory counter: job-opening + IP + email. Published job lookup, required application fields; PDF/DOC/DOCX extension and declared MIME checks; 5 MiB file check. Native create route excluded. |
| /careers/parse-resume Public: auth false | Atomic Redis counter: resume-parser + IP. Admission before body parsing; feature/key gate; one active request per process; Redis failure returns 503; multipart/header-size checks. Controller requires one PDF/DOCX file, no body fields and at most 5 MiB. |
| /product-submissions/submit Guest store visits only | CMS content API token required by trusted-magento-customer policy. product-store-visit can omit shopper identity. Memory counter: product + IP + formTag; published form, showroom and schedule validation. Other form tags require verified customer ID. |

## Client IP handling

A valid x-sunny-client-ip is accepted only with the configured shared secret, compared using timingSafeEqual. Otherwise the helper trusts x-real-ip, then ctx.ip. This is useful protection only if nginx overwrites incoming x-real-ip and the CMS origin cannot be reached around that proxy. The code comment asserts this deployment arrangement; the review did not verify it. [E1, E7]

The guest store-visit token authenticates the website server. A bot can still use any guest-facing website handler that forwards requests. Customer login is unnecessary for the proposed controls. CORS and security headers do not establish that a caller is human.

# 02 / Gaps and impact

## 1. Caller-controlled keys and process-local counters

Career applications receive a new allowance for each email on the same IP. Generic submissions split allowances by formTag; arbitrary invalid tags still allocate keys and reach form lookup, although unknown tags cannot create submissions. The in-memory map has no size cap and scans every entry on each check. Large numbers of unique keys can increase CPU and memory use. [E1-E3]

Use separate counters for IP, validated form, and normalized recipient identity. Apply the IP counter before reading identity fields. Store counters atomically in Redis with expiry so all workers share the limit. Reuse the existing Redis helper pattern after separating form and parser configuration. [E6]

## 2. Bot submissions and acknowledgement email abuse

Required fields and contact syntax checks accept plausible fabricated values. Generic reach-out, bespoke and career flows send an acknowledgement to the submitted email after creating a record. No recipient cooldown or cross-IP recipient budget was found in these senders. Attackers with rotating IPs can target one inbox. Suppress repeated acknowledgements independently of whether an enquiry is accepted. [E2-E4, E9]

## 3. Expensive work occurs before most form limits

Generic, bespoke, career and product controllers execute after body parsing. Global JSON, text and form limits are 25 MB; the default per-file parser limit is 25 MiB, while controllers later enforce 5 MiB for supported uploads. Controller rejection cannot recover resources already spent receiving/parsing a body. Enforce total bytes, file count and request deadlines before or during streaming. [E5]

The resume admission middleware is a stronger existing pattern, but its Content-Length check alone does not bound chunked bodies. The parser has a per-process busy flag, not a global cost budget. Add streamed size enforcement and a global parser concurrency/spend ceiling. [E5-E6]

## 4. Alternate writes and retries need explicit policy

Disable generated create routes for generic and bespoke submissions, or make every allowed write path enforce the same controls. Confirm Public role and website token scopes in staging. A missing permission is not currently proven. [E8]

Repeated valid requests can create additional records. Guest store visits intentionally bypass the signed-in customer clash check. Use short-lived, session-bound idempotency for retries; do not deduplicate or reveal bookings solely by an unverified email/phone. Career upload failure deletes the new record; bespoke upload failure has no equivalent compensation in the reviewed controller, so retries can leave partial records. [E3-E4, E10]

# 03 / Recommended implementation

## First release: close the reliable bypasses

Backend: replace ordinary Map counters with atomic Redis counters, apply a total IP quota independently of email/formTag, bound key length/cardinality and validate numeric settings at startup. Preserve Retry-After. Use keyed hashes for recipient identifiers and expire counters; avoid raw contact data in logs.

Operations: verify origin access is restricted, nginx overwrites client IP headers, and forwarded secrets exist only on trusted servers. Apply edge request-rate and body limits to each submission route, including the website proxy routes. Backend: exclude generic/bespoke native create, then test the actual Public role and service-token permissions.

## Second release: verify a challenge on the server

Add Cloudflare Turnstile to public forms and the guest store-visit flow. Validate its token on the server before persistence, uploads to permanent storage, email or resume extraction. Check success, expected hostname and the form-specific action. Missing, invalid or reused tokens must not reach these side effects. Tokens are single-use and expire after five minutes; clients must refresh them after expiry. [S1]

For multipart forms, perform a cheap IP admission check before reading files. Either verify a challenge before a temporary upload flow or use a short-lived, single-use admission ticket bound to the form/session and upload constraints. The current resume parser rejects extra body fields, so its challenge contract needs an explicit update.

A honeypot and a server-signed form timestamp can contribute weak spam signals. Do not hard-reject fast completion alone: autofill and assistive technology can be fast. Monitor challenge failures and preserve entered data during retriable outages.

## Suggested initial limits: tune using real traffic

| Control | Starting policy, not a measured production optimum |
|---|---|
| Submission quota | Retain 5/hour per IP per valid form initially. Add an aggregate IP ceiling (for example 20/hour), plus a short burst ceiling. Check shared-network false positives before enforcing. |
| Acknowledgement email | At most 1 per recipient + form in 15 minutes and 3/day across forms. Accept legitimate enquiries while suppressing excess acknowledgements. |
| Retry idempotency | Single operation key bound to anonymous session and payload; 10-minute retention. Atomic reservation and one saved result; reuse with changed payload is rejected. |
| Payload limits | For text-only forms, start at 64 KiB; explicit name/message limits. Upload forms: one allowed file, 5 MiB file cap and a small multipart allowance; verify file content signatures. |

Global parser cost/concurrency ceilings and edge thresholds require traffic and provider-budget data. Configure them separately from ordinary submissions. A CAPTCHA adds friction for bots but cannot guarantee spam elimination. [S1-S3]

# 04 / Verification & rollout

## Checks completed for this report

Passed 10 existing tests using node --test scripts/test-client-ip.cjs scripts/test-resume-rate-limit.cjs. Coverage includes forwarded-secret handling, Redis reply validation, timeout, Retry-After, outage behavior, busy reservation and file cleanup. Redis calls were mocked; no live Redis integration was run.

A separate local helper-level assertion confirmed that attempts 1-5 for one career IP/email are admitted, attempt 6 is blocked, and a changed email is immediately admitted. This proves counter behavior; it was not a live HTTP submission test.

## Required acceptance tests for the proposed changes

| Scenario | Required result |
| --- | --- |
| Identity rotation / workers | Changing email or formTag cannot bypass aggregate IP limits. Two workers share the same quota; restart does not clear it. Invalid tags cannot create unbounded keys. |
| Challenge / origin | Missing, replayed, expired and wrong-action/hostname tokens cause no side effects. Forged IP headers fail through the real proxy. Direct origin access is blocked. |
| Outages / retries | Redis or challenge timeout yields a retriable response without a submission or email. UI retains data. Successful request replay returns the prior result without a second write/send. |
| Uploads / permissions | Oversized chunked bodies, extra files and disguised formats fail early. Public native create is denied. Intended guest submissions still succeed without shopper login. |
| Inbox / usability | Distributed requests to the same address trigger acknowledgement suppression. Shared office IPs, mobile IP changes, autofill and accessibility flows remain usable. |

## Deployment, rollback and signals

Deploy metrics first; observe proposed aggregate limits before enforcing them. Release shared counters and route restrictions, then canary challenge enforcement by form. New counters and temporary idempotency records need TTLs; an outbox or durable idempotency design needs an additive migration and a retention policy.

On Redis/challenge outages, expensive public writes should stop with a retriable 503; do not silently disable protection. Keep the parser feature switch as an emergency stop. Rollback can disable a faulty challenge integration while retaining shared limits and edge controls. Do not reopen generated create routes.

Track accepted/rejected requests by form and reason, challenge success rate, 429/503 rates, Redis latency, queue depth, duplicate suppression, email sends per recipient hash and parser cost. Alert on abrupt acceptance spikes, provider failures or spend approaching its configured ceiling. Review conversion changes before tightening quotas.

Remaining uncertainty: production protections may be stronger or weaker than the repository. Confirm nginx/CDN rules, effective environment overrides, website relay behavior, permissions and replica count before assigning production exposure.

# 05 / Evidence & sources

Paths below are relative to D:/sunnydiamonds/sunnydiamond-cms. Line references identify the reviewed checkout.

**E1** src/utils/form-submission-rate-limit.ts:9
Defaults, in-memory counters, key construction and proxy trust.

**E2** src/api/generic-submission/controllers/generic-submission.ts:85
Generic form limiter, validation, persistence and confirmation trigger.

**E3** src/api/submissions-job-opening/controllers/submissions-job-opening.ts:82
Email-dependent career key, file validation and upload rollback.

**E4** src/api/bespoke-submission/controllers/bespoke-submission.ts:42
Bespoke limiter, image checks and write/upload ordering.

**E5** config/middlewares.ts:33
Admission ordering and global request body limits.

**E6** src/middlewares/resume-parser-admission.ts:5
Parser pre-body gate, per-process concurrency and Redis error behavior.

**E6a** src/utils/resume-parser-rate-limit.ts:80
Atomic Redis counter and shared form limit environment settings.

**E7** src/policies/trusted-magento-customer.ts:4
API-token requirement and guest form exception.

**E7a** src/api/product-submission/routes/product-submission-submit.ts:8
product-store-visit guest allowlist.

**E8** src/api/generic-submission/routes/generic-submission.ts:3
Generated core router; bespoke equivalent also inspected.

**E9** src/utils/reach-out-confirmation-email.ts:10
Acknowledgement sender; career-application-email.ts and custom-creation-email.ts also inspected.

**E10** src/utils/store-visit-clash.ts:17
Guest exclusion from customer clash protection.

## External guidance checked on 7 October 2026

S1 - Cloudflare: server-side Turnstile validation
https://developers.cloudflare.com/turnstile/get-started/server-side-validation/

S2 - OWASP: Denial of Service Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html

S3 - OWASP: File Upload Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html